

Data Structures and Algorithms - Sprint 3

Gabriel Augusto da Silva - RM 567057

Leonardo Kenji Kubo Barboza - RM 567518

Lucas Gabriel Alvarenga e Meireles - RM 567305

Lucas Koiti Uyeno de Souza - RM 568128

Lucas Morio Ikeda - RM 567616

RELATÓRIO TÉCNICO

Estruturas utilizadas:

Fila (FIFO - First In, First Out): duas filas foram utilizadas para organizar os pacientes, uma para casos comuns e outra para casos prioritários. Os pacientes são chamados em ordem de chegada, com a fila prioritária sendo verificada primeiro.

Operações da Fila:

- `inicializarFila()`: inicializa a fila vazia;
- `filaVazia()`: verifica se a fila está vazia;
- `enqueue()`: insere um elemento no final da fila;
- `dequeue()`: remove um elemento da início da fila;
- `exibirProximosPacientes()`: exibe os próximos N pacientes presentes na fila, N sendo recebido como um parâmetro na chamada da função;
- `liberarMemoriaFila()`: percorre a fila inteira e libera a memória de cada nó.

Pilha (LIFO - Last In, First Out): uma pilha foi utilizada para armazenar o histórico das consultas realizadas.

Operações da Pilha:

- `inicializarPilha()`: inicializa a pilha vazia;
- `pilhaVazia()`: verifica se a pilha está vazia;
- `push()`: insere um elemento no topo da pilha;
- `pop()`: remove um elemento do topo da pilha;
- `peek()`: olha qual é o elemento no topo da pilha;

- `exibirHistorico()`: exibe as últimas N consultas presentes no histórico, N sendo recebido como um parâmetro na chamada da função;
- `liberarMemoriaPilha()`: percorre a pilha inteira e libera a memória de cada nó.

Outras operações:

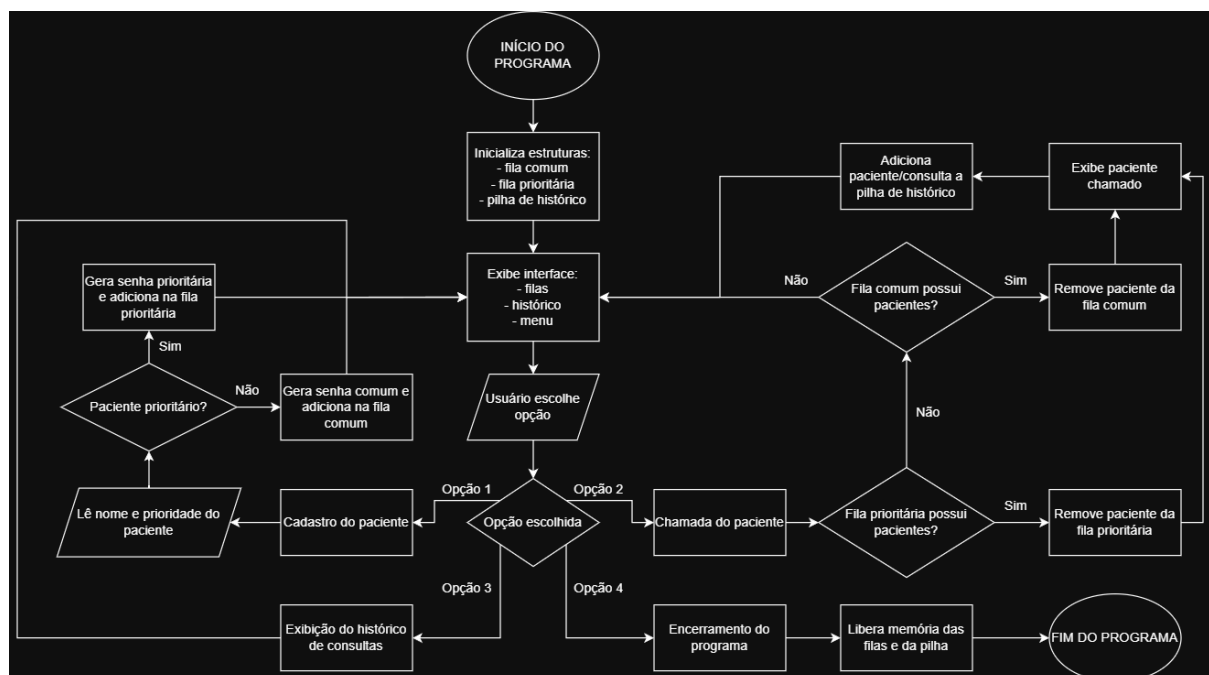
- `exibirInterface()`: exibe a interface no output para utilização do usuário.

Funcionamento:

O sistema apresenta uma interface simples mostrando os próximos 4 pacientes em cada fila. quatro opções ao usuário:

1. Cadastro de pacientes: o usuário digita o nome do paciente e escolhe sua prioridade, 0 é comum e 1 é prioritário. O paciente receberá uma senha e será adicionado à sua respectiva fila conforme a prioridade.
2. Chamada de paciente: as filas são verificadas, a fila prioritária tomando precedência. Caso haja um paciente esperando, ele será chamado, sendo removido da fila e adicionado a pilha do histórico de consultas.
3. Exibição do histórico de consultas: exibe as últimas 10 consultas no histórico.
4. Encerramento do programa: se o usuário digitar 0, quarta opção, o programa será encerrado.

Diagrama:



Complexidade das operações:

- `inicializarFila()`: complexidade $O(1)$, inicializa os ponteiros *frente* e *fim* como NULL;
- `filaVazia()`: complexidade $O(1)$, verifica se o ponteiro *frente* é igual a NULL, se a fila está vazia;
- `enqueue()`: complexidade $O(1)$, cria um novo nó e o adiciona ao final da fila utilizando o ponteiro *fim*. A operação é constante pois o sistema possui acesso ao último elemento da fila;
- `dequeue()`: complexidade $O(1)$, remove o elemento da frente da fila e atualiza o ponteiro *frente*. A operação é constante pois a remoção é feita diretamente no início da fila;
- `exibirProximosPacientes()`: complexidade $O(n)$, percorre a fila exibindo os próximos N pacientes. No pior dos casos, percorre a fila inteira;
- `liberarMemoriaFila()`: complexidade $O(n)$, percorre toda a fila chamando `dequeue()` sucessivamente até que ela esteja vazia;
- `inicializarPilha()`: complexidade $O(1)$, inicializa o ponteiro *topo* como NULL;
- `pilhaVazia()`: complexidade $O(1)$, verifica se o ponteiro *topo* é igual a NULL;
- `push()`: complexidade $O(1)$, insere um novo nó no topo da pilha. A operação é constante pois a inserção ocorre diretamente no topo da pilha;
- `pop()`: complexidade $O(1)$, remove o elemento do topo da pilha e atualiza o ponteiro *topo*. A operação é constante pois a remoção é feita diretamente no topo da pilha;
- `peek()`: complexidade $O(1)$, acessa diretamente o elemento do topo da pilha;
- `exibirHistorico()`: complexidade $O(n)$, percorre a pilha exibindo as últimas N consultas. No pior dos casos, percorre a pilha inteira;
- `liberarMemoriaPilha()`: complexidade $O(n)$, percorre toda a pilha chamando `pop()` sucessivamente até que ela esteja vazia;
- `exibirInterface()`: complexidade $O(n)$, exibe informações das filas e da pilha chamando `peek()` e `exibirProximosPacientes()`. No pior dos casos, a operação possui complexidade linear.

Links:

https://github.com/koitilucas/DSA_Sprint3

https://youtu.be/0NH8zII__U